MIT 8103 – ADVANCED DATABASE SYSTEMS

FINAL PORTFOLIO SUMMARY AND TECHNICAL REFLECTION

Student Name: Ejereje Erere Oyibo

Student ID: 301908132

Case Study: Kart Direct Courier E-Commerce and Delivery Management System

1. Introduction and Selected Organization

This document provides a consolidated summary of the five practical portfolio activities completed for MIT 8103 – Advanced Database Systems. The selected case study is Kart Direct Courier E-Commerce and Delivery Management System, a simulated Nigerian e-commerce and courier business. The system manages customers, addresses, product categories, products, orders, order items, payments, couriers, deliveries, and delivery-status history.

The work was developed using PostgreSQL as the relational database platform and MongoDB as the NoSQL platform. The five portfolios address database design and modelling, query processing and optimization, transactions and concurrency, NoSQL data modelling, and distributed database concepts. The individual portfolio reports and supporting screenshots remain in their corresponding folders in the GitHub repository.

2. Summary of Portfolio Activities

Portfolio 1 – Database Design and Modelling

A PostgreSQL database was designed and implemented with ten related tables: CUSTOMERS, CUSTOMER_ADDRESSES, CATEGORIES, PRODUCTS, ORDERS, ORDER_ITEMS, PAYMENTS, COURIERS, DELIVERIES and DELIVERY_STATUS_HISTORY. A relational schema and entity-relationship diagram were prepared to show the entities, attributes, and relationships. Primary keys, foreign keys, unique constraints, CHECK constraints, and default values support data integrity. The report also explains First, Second, and Third Normal Forms. Evidence includes database and table creation, sample-data verification, record counts, and a relational JOIN query.

Portfolio 2 – Query Processing and Optimization

SQL queries retrieved and sorted orders and calculated order totals by joining orders, customers, and order_items. SUM() and GROUP BY were used to calculate totals. EXPLAIN ANALYZE was used to inspect PostgreSQL execution plans. An index named idx_orders_order_date was created

and verified, and a date-filtered query was tested. PostgreSQL continued to choose a sequential scan for the small sample dataset, demonstrating that creating an index does not guarantee that the optimizer will use it. The report notes that larger datasets would be needed for stronger performance conclusions.

Portfolio 3 – Transactions and Concurrency

Transaction control was demonstrated in PostgreSQL using the orders table and order 5. The ROLLBACK exercise changed the order status within a transaction and then cancelled the change, restoring the previously committed value. The COMMIT exercise demonstrated that an update could be saved permanently. A two-session test examined the default READ COMMITTED isolation behaviour: while Session A had an uncommitted update, Session B continued to read the last committed status; after Session A committed, Session B could see the new status. Screenshots document the rollback, commit, and concurrent-read tests.

Portfolio 4 – NoSQL and Advanced Data Models

A MongoDB database named kart_direct_courier_nosql and an Orders collection were created using MongoDB Community Server and MongoDB Compass. Two sample order documents were inserted. Each document contains order information, an embedded delivery document and an items array containing product identifiers and quantities. MongoDB queries demonstrated retrieval by order status, embedded delivery status, and product identifier within the items array. The report compares MongoDB's flexible document model with PostgreSQL's relational structure and explains the usefulness of embedding related order and delivery information when those details are commonly retrieved together.

Portfolio 5 – Distributed and Cloud Database Exercise

PostgreSQL logical replication was configured between publisher database kart_direct_courier and subscriber database kart_direct_courier_sub. The public.orders table was included in orders_publication, with orders_subscription configured on the subscriber. The initial data copy transferred five order records. Replication status and LSN values were checked. Order 5 was changed on the publisher from Confirmed to Processing and verified on the subscriber; it was then restored to Confirmed and verified again. This demonstrates publisher-to-subscriber logical replication. Both databases were hosted on the same PostgreSQL server/cluster, so this setup does not protect against failure of that server. Sharding, availability, consistency and production deployment considerations were discussed conceptually rather than implemented.

3. Overall Learning and Technical Reflection

The five activities provided practical experience of how different database technologies and techniques support the information needs of one business. The first portfolio established the relational foundation, while the second showed how queries, execution plans and indexes can be used to retrieve and analyze data. The third made transaction behaviour practical by showing the effects of COMMIT, ROLLBACK and isolation across two sessions.

The MongoDB exercise demonstrated that the same business information can be represented using documents, embedded objects and arrays. The logical replication exercise introduced keeping a selected table synchronized between a publisher and subscriber. It also highlighted the limits of a test environment: a subscriber on the same server demonstrates replication, but it is not equivalent to a separate high-availability deployment.

A key lesson is that database decisions should be based on application requirements and evidence from testing. An index should be evaluated using execution plans and suitable data volumes rather than assumed to improve every query. Replication in production should be accompanied by infrastructure separation, monitoring, backups, and tested recovery arrangements.

4. Evidence, Repository and Submission Structure

The GitHub repository contains five separate portfolio folders. Each folder holds its corresponding portfolio report and available supporting evidence, including screenshots and practical outputs. The Portfolio 5 report includes an appendix of logical replication screenshots. The individual reports should remain available separately so the assessor can inspect the detailed implementation evidence for each activity.

GitHub repository:

https://github.com/eejereje8132/MIT8103-Advanced-Database-Systems-CA

Repository folders: Portfolio 1~ Database Design, Portfolio 2 ~ Query Optimization, Portfolio 3~ Transactions Concurrency, Portfolio 4 ~NoSQL, and Portfolio 5~ Distributed Cloud. The repository README provides an overview of the case study, portfolio structure, and tools used.

5. References

PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation. https://www.postgresql.org/docs/

MongoDB, Inc. (n.d.). MongoDB manual. https://www.mongodb.com/docs/manual/

6. AI Use Declaration

I used ChatGPT as a learning and drafting aid to help explain database concepts, clarify areas where I needed further understanding, and organize parts of the portfolio documentation. I carried out the practical database exercises using PostgreSQL, pgAdmin 4, MongoDB Compass and MongoDB Shell, and reviewed the results against the screenshots and outputs included in my portfolio. I remain responsible for the work submitted and for understanding the database operations demonstrated.